PAGINA WEB

ReactJS
Introducción

- ¿Qué es ReactJS?
- Componente
  - ¿Qué es un componente?
  - ¿Como usar un componente?
  - Exportar e importar un componente
- JSX
  - Las reglas de JSX
  - Insertando expresiones en JSX
  - Especificando atributos con JSX
- Prop

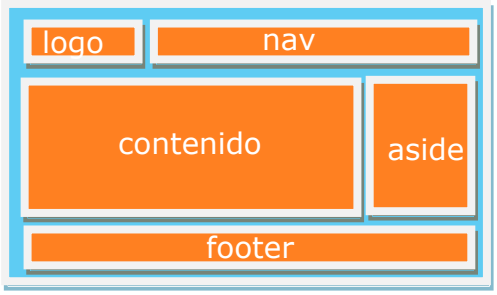
Profesora:
Elizabeth Gisela Oviedo Luna

REACT JS

ReactJS o simplemente React es una biblioteca o **librería JavaScript** de código abierto para construir interfaces de usuario de forma sencilla mediante componentes interactivos y reutilizables.

Interfaces de usuario serían los elementos que los usuarios ven e interactúan en la pantalla. Biblioteca son un conjunto de funciones útiles (API) para crear una interfaz de usuario.

React está basado en un paradigma llamado **programación orientada a componentes** en el que cada componente es una parte de la interfaz de usuario. Por ejemplo, un componente para la barra de navegación, otro para el pie de página, otro para el contenido principal, etc.



Estos componentes son reutilizables y se combinan para crear componentes mayores hasta configurar una web completa. No tenemos que repetir el código reiterativo. Sólo tendríamos que crear su lógica e importar el componente en cualquier parte del código donde se necesite.

Estas piezas se crean usando una sintaxis llamada **JSX** (JavaScript XML), permitiendo escribir HTML, y opcionalmente CSS, dentro de objetos JavaScript.

Es la forma de tener HTML con toda la funcionalidad de JavaScript y el estilo gráfico de CSS centralizado y listo para ser abstraído y usado en cualquier otro proyecto.

Posee un **DOM virtual.** Una copia del objeto DOM que primero actualiza y vuelve a renderizar nuestras páginas cuando se realizan cambios; luego compara el estado actual con el DOM original para mantenerlo sincronizado con los cambios. Esto hace que la renderización de las páginas sea más rápida.

Debemos considerar al crear aplicaciones:

- **Interfaz de usuario**: cómo los usuarios verán e interactuarán con su aplicación.
- **Enrutamiento**: cómo se navega entre diferentes partes de su aplicación.
- **Obtención de datos**: dónde se encuentran sus datos y cómo obtenerlos.
- **Renderizado**: cuándo y dónde renderizas contenido estático o dinámico.
- **Integraciones**: qué servicios de terceros utiliza (CMS, autenticación, pagos, etc.) y cómo se conecta a ellos.
- **Infraestructura**: donde implementa, almacena y ejecuta el código de su aplicación.
- **Rendimiento**: cómo optimizar su aplicación para los usuarios finales.
- **Escalabilidad**: cómo se adapta su aplicación a medida que crecen su equipo, sus datos y su tráfico.

- **Experiencia del desarrollador**: la experiencia de su equipo en la creación y el mantenimiento de su aplicación.

Para cada parte de su aplicación, deberá decidir si creará una solución usted mismo o utilizará otras herramientas, como paquetes, bibliotecas y marcos.

Ventajas:

- **Fácil de aprender** y comprender: siempre que se conozcan bien los requisitos previos. Puede integrar a otros miembros de tu equipo que no sean programadores en el desarrollo. Es "fácil" leer el código si ellos están acostumbrados a leer HTML.
- **Componentes reutilizables**: Cada componente en React tiene su propia lógica que puede reutilizarse en cualquier parte de la aplicación. Esto reduce la necesidad de reescribir el mismo código varias veces.
- **Mejora del rendimiento**: Con el DOM virtual de React, la renderización de las páginas puede hacerse más rápidamente. Al utilizar una biblioteca de enrutamiento como React Router, tendrás diferentes páginas renderizadas sin necesidad de recargar la página.
- **Ampliamente extensible**: React es una librería que sólo renderiza la interfaz de usuario de nuestra aplicación. Depende del desarrollador elegir con qué herramientas trabajar.

Al visitar una página web, el servidor devuelve un archivo HTML al navegador; este lo lee y construye el modelo de objetos de documento (DOM). React actualizará el DOM de acuerdo a como lo programemos que se maneje la interfaz de usuario.

¿Qué es un componente?

Un componente es una pieza independiente de nuestra interfaz:

- Es aislado o bloque separado.
- Reutilizable.
- Define la estructura (html), el estilo (css) y el comportamiento (js).
- Es una función de JavaScript que puedes agregar con marcado de HTML
- Sus nombres deben comenzar con una letra mayúscula o no funcionarán.
- Pueden ir desde algo básico como un botón a una página completa

Ejemplos de componente:

```
export default function ArticuloA() {
  return (
    <article>
      <h2>Titulo</h2>
      <p>Parrafo</p>
    </article>
  )
}
```

Analizando:

- *export default*: sintaxis estándar de JavaScript. Exporta el componente de un archivo para luego poder importarla desde otros archivos.
- *function*: define una función de JavaScript con el nombre ArticuloA.
- *código de la función:* Es código de JSX que le permite agregar marcado dentro de JS.
- *return*: si es una sentencia de una sola línea puede ir sin paréntesis; en caso de utilizar más de una línea se debe encerrar entre paréntesis si no las ignorará.

```
export default function ImagenFoto() {
  return <img src="foto.jpg" alt="imagen" />
}
```

¿Cómo usar un componente?

Si tenemos el componente

```
function BotonAgregar() { return (<button>Agregar</button>); }
```

Lo utilizamos nombrándolo de la siguiente forma: **<BotonAgregar/>**

Siempre empieza con mayúscula que denota ser un componente de React.

Los nombres de los componentes de React siempre deben comenzar con mayúscula, mientras las etiquetas HTML deben estar minúsculas.

Los componentes se pueden combinar, ordenar y anidar para diseñar páginas completas.

```
export default function NoticiaA() {
  return (
    <section>
      <h2>Noticia ABC</h2>
      <ImagenFoto />
      <ArticuloA />
    </section>
  )
}
```

React distingue entre las etiquetas:

- Minúsculas las toma como etiquetas HTML: section y h2.
- Mayúsculas las toma como componente: ImagenFoto y ArticuloA

Al final lo que ve el navegador es:

```
<section>
  <h2>Noticia ABC</h2>
  <img src="foto.jpg" alt="imagen" />
  <article>
    <h2>Titulo</h2>
    <p>Parrafo</p>
  </article>
</section>
```

Los componentes pueden incluir otros componentes, pero nunca debes anidar sus definiciones.

```
export default function ArticuloB() {
  return (
    <article>
        //Error!!!
        function Imagen(){
          return <img />
        }
      <p>Parrafo</p>
    </article>
  )
}
```

La forma correcta es:

```
export default function ArticuloB() {
  return (
    <article>
      <ImagenB />
      <p>Parrafo</p>
    </article>
  )
}

function ImagenB() {
  return <img src="fotoB.jpg" alt="imagen" />
}
```

Exportar e importar un componente

Los componentes como son reutilizables para modularizarlos se guardan en diferentes archivos.

El archivo del componente raíz en React de una página generalmente se denomina App.js. Sin embargo, esto es configurable por lo tanto podría ser otro archivo. Por ejemplo, si utilizamos un framework como Next.js con enrutamiento basado en archivos su componente raíz será diferente para cada página.

Exportar

Para exportar un componente existen dos tipos diferentes de exportación:

- por nombre:
  - Formato:

```
export function ExportNombreA() {
  return //...
}
```

  - En un módulo se pueden tener varias exportaciones nombradas
  - Cuando se importa, es obligatorio usar el mismo nombre.

- por defecto (default).

```
export default function ExportDefecto() {
  return //..
}
```

  - En un módulo solo puede haber una exportación por defecto
  - Cuando se importa puede ser importado con cualquier nombre.

Importar

Utilizar la sentencia **import** para importar funciones que han sido exportadas desde otro módulo

El módulo desde el que se importa se nota entre comillas simples o dobles, como una ruta relativa o absoluta al archivo que contiene el módulo; la extensión del archivo puede omitirse o dejarla explícitamente.

Se puede renombrar un componente al importarlo.

```
import { ExportNombreA } from "../modA.js";
import  ExportDefecto  from "../modA.js";
import { ExportNombreA, ExportNombreB as B } from "../modA.js";
import ExportDefecto, { ExportNombreA, ExportNombreB as B } from "../modA.js";
```

JSX

Es una **extensión de sintaxis de Javascript**, que se parece a HTML.

React no requiere usar JSX, pero facilita el desarrollo de aplicaciones React. Es útil como ayuda visual cuando trabajan con interfaz de usuario dentro del código JavaScript.

Para aprender JSX, teniendo previamente HTML y JavaScript solo es tener en cuenta las siguientes las tres reglas de JSX

Las reglas de JSX

1° Regla: Devolver un único elemento raíz

Para devolver varios elementos de un componente, hay que envolverlos con una única etiqueta principal para que una función pueda devolver un objeto y no múltiples objetos; que por lo general puede ser .

Ejemplo: Puede usar un :

```
<div>
    <h1>Titulo</h1>
    <p>Párrafo A</p>
    <img src=".." alt="imagen"/>
    <p>Otro párrafo</p>
</div>
```

Fragmento: En caso de no querer agregar una etiqueta extra como se puede utilizar una etiqueta vacía <> </> a la que se le llama Fragmento. Los fragmentos te permiten agrupar cosas sin dejar ningún rastro en el árbol HTML del navegador.

```
<>
  <h1>Titulo</h1>
  <p>Párrafo A</p>
  <img src=".." alt="imagen" />
  <p>Otro párrafo</p>
</>
```

2° Regla: Cierra todas las etiquetas

Las etiquetas se deben cerrar explícitamente:

- Las etiquetas que no tienen un par de cierre como <img …> deben convertirse en <img …/>

```
<img src=".." alt="imagen" />
```

- Las etiquetas que vienen de a pares como deben tener su par .

```
<p>párrafo</p>
```

3° Regla: Notación camelCase

JSX se convierte en JavaScript y este tiene limitaciones en cuanto a los nombres de variables. Sus nombres no pueden contener guiones ni ser palabras reservadas como class; class se vuelve className en JSX y tabindex se vuelve tabIndex.

```
<p className="prinpipal">Párrafo A</p>
```

Los componentes de React agrupan la lógica de representación junto con el marcado porque están relacionados.

JSX es similar a HTML, con algunas diferencias, puede utilizarse un convertidor como https://transform.tools/html-to-jsx si es necesario. Los mensajes de error a menudo le indicarán la dirección correcta para corregir su marcado.

Permite que React muestre mensajes de error o advertencia más útiles.

Insertando expresiones en JSX

Es posible usar JavaScript dentro del marcado HTML en JSX utilizando llaves.

Utilizando llaves

Se puede usar:

- Como texto directamente dentro de una etiqueta JSX:
  - Funciona -><h1>{name} </h1>
  - Error NO Funciona -> <{etiqueta}>María</{etiqueta}>.
- Como atributos inmediatamente después del signo =:
  - Funciona-> src={fuente}
  - NO funciona-> src="{ fuente }" , utilizará la cadena "{fuente}".

Ejemplo: Declaramos la variable nombre y la utilizamos en JSX entre llaves.

```
let nombre="Miguel";
let saludo= <h1>Hola {nombre}!</h1>;
```

Cualquier expresión de JavaScript funcionará entre llaves, incluidas llamadas a funciones

Ejemplo: Llamamos la función de JavaScript, nombreCompleto(usuario), dentro de un elemento <h1>.

```
function nombreCompleto(persona)
{ return usuario.nombre +" "+usuario.apellido;}

let usuario={nombre:'Rita', apellido:'Fernandez'}
let saludo= <h1>Hola {nombreCompleto(usuario)}!</h1>;
```

Especificando atributos con JSX

Cuando desea pasar un atributo de cadena a JSX, lo coloca entre comillas simples o dobles para especificar strings literales como atributos o puedes usar llaves para insertar una expresión JavaScript en un atributo:

```
let eltoImg= <img src={usuario.avatar} alt="persona de frente"/>;
```

En React, especificas una clase de CSS con className. Funciona de la misma forma que el atributo class de HTML:

<button className="avatar" />

Suponiendo un html

```
<body>
  <div id="contenedor"></div>
</body>
```

En React:

```
function BotonAgregar() { return (<button>Agregar</button>); }
let ficha = function () { return (<div>{saludo}{eltoImg}</div>); }
function nombreCompleto(persona)
{ return usuario.nombre + " " + usuario.apellido; }

let usuario = { nombre: 'Rita', apellido: 'Fernandez', avatar: 'foto.jpeg' }
let saludo = <h1>Hola {nombreCompleto(usuario)}!</h1>;
let eltoImg = <img src={usuario.avatar} alt="persona de frente" />;

const lugar = createRoot(document.getElementById('contenedor'));
lugar.render(
  <div>
    {ficha()}
    <BotonAgregar />
  </div>
);
```

Hola Rita Fernandez!

Agregar

Además se puede utilizar doble llaves {{y }}; no es una sintaxis especial: es un objeto JavaScript escondido entre llaves JSX o el estilo on-line css.

```
function BotonAgregar() {
  return (
    <button style={{ borderRadius: '30%', backgroundColor:'#aaffaf' }}>
      Nuevo
    </button>);
  }
```

Nuevo

En el ejemplo se pasa un parámetro a la función, esto es un PROP.

Prop

Es un objeto de JavaScript que es pasado como un parámetro a un componente.

Props es sólo una forma más corta de decir propiedades. Son útiles cuando desea que el flujo de datos en su aplicación sea dinámico.

Los props son solo de lectura, esto significa que no debes cambiarles su valor.

Los valores de los props pueden ser: un objeto, array, cadena, número, función, incluso componentes o elementos.

```
function Ficha(datos) {
  return (
    <div>
      <h3>{datos.nombre}</h3>
      <img src={datos.foto} alt={datos.nombre} width='200px'/>
      <p>Cargo: {datos.cargo}</p>
    </div>
  )
}
function Staff() {
  return (
    <section>
      <Ficha nombre="Julia" foto="julia.jpg" cargo="Gerente" />
      <Ficha nombre="Lorenzo" foto="lorenzo.jpg" cargo="Ventas" /:
    </section>
  )
}
```

Julia

Cargo: Gerente

Lorenzo

Cargo: Ventas

Otra alternativa es desestructurar:

```
function Ficha({persona, cargo }) {
  return (
    <div>
      <h3>{persona.nombre}</h3>
      <img src={persona.foto} alt={persona.nombre} width='200px'/>
      <p>Cargo: {cargo}</p>
    </div>
  )
}

function Staff() {
  return (
    <section>
      <Ficha persona={{nombre:"Julia", foto:"julia.jpg"}} cargo="Gerente" />
      <Ficha persona={{nombre:"Lorenzo", foto:"lorenzo.jpg"}} cargo="Ventas" />
    </section>
  )
}
```

Se pueden especificar valores por defecto:

```
function Ficha({nombre, foto="nn.jpg", cargo="Ventas" }) {
  return (
    <div>
      <h3>{nombre}</h3>
      <img src={foto} alt={nombre} width='200px'/>
      <p>Cargo: {cargo}</p>
    </div>
  )
}
function Staff() {
  return (
    <section>
      <Ficha nombre="Julia"  cargo="Gerente" />
      <Ficha nombre="Lorenzo" foto="lorenzo.jpg" />
    </section>
  )
}
```